Contents

1 Introduzione al corso **2**

2 Lezione del 08-03 **3**

2.1 I sistemi operativi . . . 3

2.2 Sistemi mono e multi programmati . . . 3

2.3 Sistemi paralleli . . . 4

2.4 Sistemi distribuiti . . . 4

2.5 Ambiente di elaborazione (Client-Server o Peer-to-Peer) . . . 4

2.6 Cloud computing . . . 4

2.7 Sistemi mobile . . . 4

2.8 Avvio di un sistema operativo nel dettaglio . . . 5

2.9 Tabella degli interrupt . . . 6

2.10 Memoria centrale . . . 6

2.11 Memoria cache . . . 6

2.12 Memoria di massa . . . 6

2.13 I processi . . . 7

2.14 Activity . . . 8

2.15 Politiche utilizzate dallo scheduler . . . 9

2.16 Sistema mono e multi tasking . . . 9

2.17 Throughput (utilizzo della CPU) . . . 10

2.18 Overhead . . . 10

3 Lezione del 12-03 **10**

3.1 Sincronizzazione dei processi 10

3.2 Comunicazione con i dispositivi 12

3.3 File system 12

3.4 La shell dei comandi 13

3.5 La memoria virtuale 13

3.6 Memoria virtuale in Linux 14

3.7 Spazio di indirizzi di un processo 15

4 Lezione del 15-03 **17**

4.1 Swapping 17

4.2 Memory Arbiter 17

4.3 La RAM nei sistemi UNIX 18

4.3.1 Risultati desiderati 18

4.3.2 Stack & heap 18

4.4 Memory Management Unit (MMU) 20

4.5 Segmentazione 20

4.6 Indirizzo logico 20

4.6.1 Segment selector 20

4.6.2 Offset 21

4.6.3 Descrittore di segmento 21

4.6.4 Traduzione degli indirizzi logici in indirizzi fisici 22

4.6.5 Privilegi di esecuzione 23

4.6.6 Tabelle dei Descrittori di Segmento 23

4.7 Segmentazione in sistemi LINUX 24

4.7.1 I segmenti in Linux . . . 25
4.7.2 Spazio degli indirizzi lineari . . . 25
4.7.3 Pagina di indirizzi lineari . . . 25
4.7.4 Segmentazione vs Paginazione . . . 25
4.7.5 Page . . . 25
4.7.6 Tipi di Paginazione . . . 26
4.7.7 TLB . . . 27

5 Lezione del 19-03 **27**

5.1 La programmazione di sistema . . . 27
5.1.1 System call . . . 27
5.1.2 Fork . . . 28
5.1.3 Processo Zombie . . . 28
5.1.4 Esempio . . . 28
5.2 Gestione dei processi in Linux . . . 28
5.3 Process Descriptor . . . 29
5.4 Stati da un processo . . . 29
5.5 Identità di un processo . . . 29
5.6 Relazioni fra processi . . . 29
5.7 Le liste di processi RUNNING . . . 29
5.8 I file hash . . . 30
5.8.1 Funzione di accesso h . . . 30
5.8.2 Divisione . . . 30
5.8.3 Risoluzione delle Collisioni . . . 30

5.8.4 Tabella Hash per la runqueue 30

5.9 Organizzazione dei processi 31

5.9.1 Wait queue 31

5.9.2 Struttura di una Wait queue 31

5.10 Duplicazione delle risorse 31

5.10.1 Chiamate di sistema in Linux 32

5.11 Bomba fork 32

5.12 Terminazione di un processo 32

5.13 I thread 33

5.13.1 Kernel-level e user-level thread 33

5.13.2 Clone() 34

5.13.3 Pthreads 34

+TITLE: Sistemi operativi II

1 Introduzione al corso

`Ricevimento` (da concordare almeno 24 ore prima) → Venerdì 10-12 (chat privata su teams)

`Email` → silviobarra@unina.it con oggetto **[INFO-S02]** (iscritto al *anno di corso* e *corso di laurea* SO2 esame facoltativo o obbligatorio)

Obiettivi

- Scelte progettuali ed implementative di un reale sistema operativo a base linux
- Sistemi distribuiti e del cloud computing
- Sistemi operativi ad uso mobile
- IoT

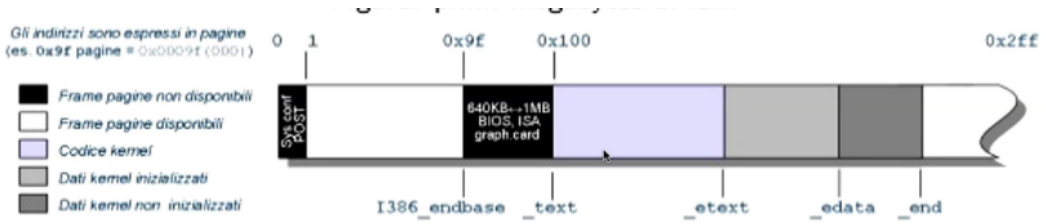
Prove intercorso:

- Domande sul sistema linux
- Domande sui sistemi mobile

2 Lezione del 08-03

2.1 I sistemi operativi

Le componenti di un sistema di calcolo è così definita:



Un **sistema operativo** è difficile da definire (in base alle grandi variazioni che presenta). Una definizione è quella di definirlo come un insieme di software atti a gestire l'hardware sottostante.

Può essere visto come un allocatore di risorse:

- memoria;
- dispositiv di `I/O`;
- processi

Può essere visto come un programma di controllo:

- gestore dei programmi di sistema ed applicativi;
- verifica del corretto funzionamento dei dispositivo di `I/O`

2.2 Sistemi mono e multi programmati

Un sistema è detto **mono-programmato** quando in ogni momento è presente una sola **immagine** (pacchetto processo + dati)

Con l'introduzione del concetto di **spooling**, nascono i primi sistemi **multi-programmati** (il sistema è in grado di eseguire più job in concorrenza). Ciò comporta di tener conto della gestione delle risorse ad essi dedicate:

- Tempo di calcolo
- Memoria
- `I/O`

In questi sistemi si ha il aso di sistemi multi-utenti. Di conseguenza nascono i sistemi **time-sharing** (nascita dei primi PC). Ogni processo utilizza periodicamente un intervallo di tempo prestabilito (un **quanto**)

2.3 Sistemi paralleli

Si arriva alla nascita dei sistemi **Paralleli** dotati di più CPU :

- Symmetric multiprocessing (SMP) → ogni processore esegue una copia identica del SO
- Asymmetric multiprocessing (AMP) → a ciascun processore è assegnato un task specifico; il processore **master** schedua e allora il lavoro per i processori **slave**

2.4 Sistemi distribuiti

Ripartiscono l'elaborazione su diversi processori. Ciascun processore ha una memoria locale e i processori comunicano con gli altri attraverso di comunicazione quali bus ad alta velocità o rete. La rete può essere:

- Locale (LAN);
- Geografica (WAN);
- Metropolitana (MAN);
- Small (SAN).

2.5 Ambiente di elaborazione (Client-Server o Peer-to-Peer)

Esistono due principali tipi di server:

- server per l'elaborazione;
- file server

Nei sistemi Peer-to-Peer invece ciascun pc può essere server o client

- Il carico è suddiviso su più server (si evitano i sovraccarichi);
- Necessita una politica di gestione
- Nuova entità (**load balancer**)

2.6 Cloud computing

Paradigma di uso delle risorse computazionali attraverso il cloud (memorizzazione, archiviazione ed elaborazione dati)

2.7 Sistemi mobile

Il sistema operativo in questo caso ha orientamento specifico (a causa delle dimensioni del dispositivo)

2.8 Avvio di un sistema operativo nel dettaglio

All'avvio (o al reset) di un sistema il programma di **bootstrap** presente sul firmware inizializza i componenti del sistema, carica ed avvia il SO, che rimane in attesa di:

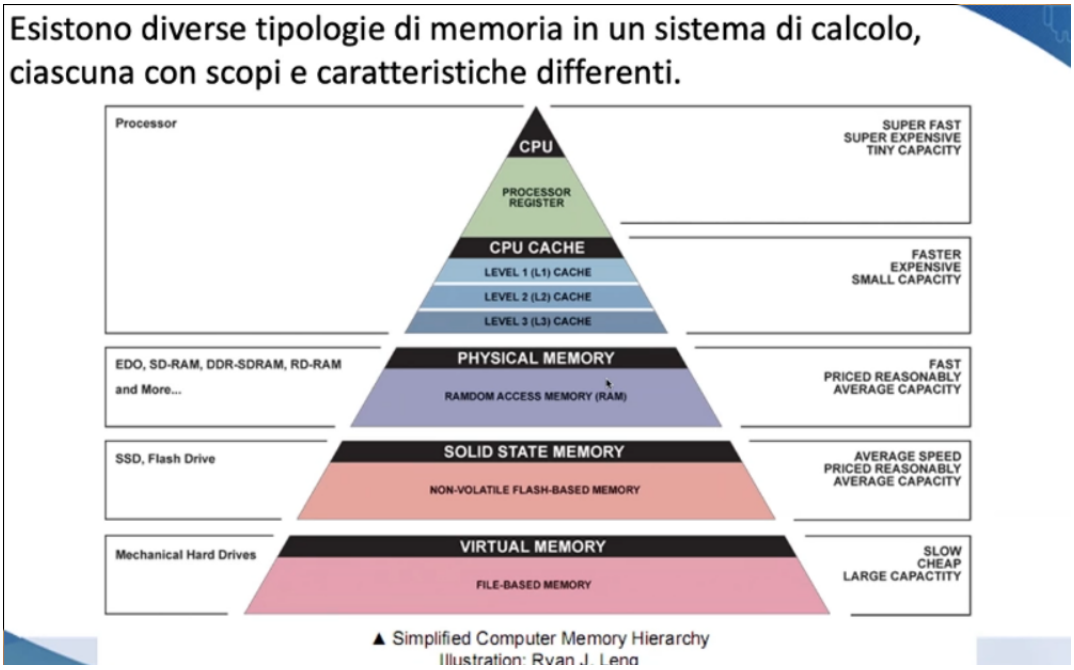
- Un programma applicativo da eseguire;
- Un interrupt → segnale generato da un controller di dispositivo o da un programma e invaito direttamente alla CPU, tramite bus;
- una exception → segnale generato da un programma a seguito di una condizione eccezionale di errore o di una specifica richiesta tramite system call.

La sequenza di avvio è:

Accensione → POST → controllo hard disk e Plug & Play → Avvio del bootloader → Avvio del SO

2.9 Tabella degli interrupt

Gli interrupt vengono gestiti mediante una strategia basata su una tabella



2.10 Memoria centrale

È rappresentata ccome un vettore di parole machina dalle centinaia di migliaia all'ordine dei miliardi di parole. Nella memoria centrale vengono caricati programmi e i dati

2.11 Memoria cache

Memoria molto veloce con lo scopo di ridurre al minimo gli accessi ad unità di memoria più lente. Bisogna garantire la coerenza delle informazioni memorizzate in una o più unità di memoria cache (il cui grado di complessità aumenta a seconda del tipo di sistema)

2.12 Memoria di massa

Divise in secondarie (dischi magnetici, solidi) e terziarie (dischi ottici, CD, DVD, nastri). Le responsabilità del SO rispetto alla gestione delle memorie di massa sono:

- gestione dello spazio libero;
- assegnazione dello spazio;
- scheduling del disco.

I dati vengono scritti nelle memoria mediante file

2.13 I processi

Un processo può essere definito come un programma di elaborazione o **job**. Un processo è un entità attiva, dotata di un program counter. La sua esecuzione è sequenziale e necessita di determinate risorse:

- CPU
- memoria
- dispositivi `I/O`;
- file

Il SO deve garantire alcune operazioni fondamentali per i processi:

- creazione e cancellazione di processi;
- sospensione e ripristino di processi;
- sincronizzazione di processori;
- comunicazione fra processi;
- soluzioni per lo stato di **deadlock** di processi (questo compito spetta al kernel attraverso lo **scheduler**)

Gli stati di un processo

Ci si può trovare in 2 stati:

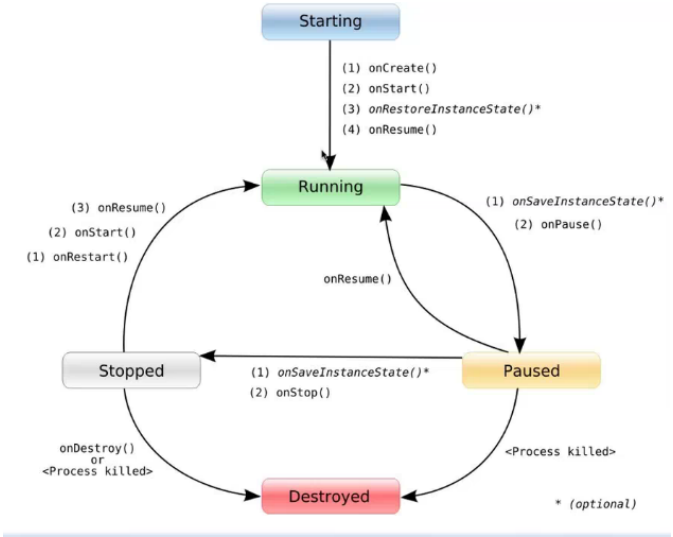
- attesa;
- pronto;

In ogni instante di tempo vi è un solo stato attivo

2.14 Activity

Una **Activity** è caratterizzata da un ciclo di vita ed in ogni istante si trova in uno dei seguenti stati:

- `Active/Resumed` (running) → essa è visibile e può ricevere input dall'utente;
- Paused → essa è parzialmente visibile a causa di un'altra activity in foreground, non può ricevere input dall'utente (*es* è in attesa a causa di un alert dialog presente al di sopra di essa);
- Stopped → essa non è visibile (resta in background)

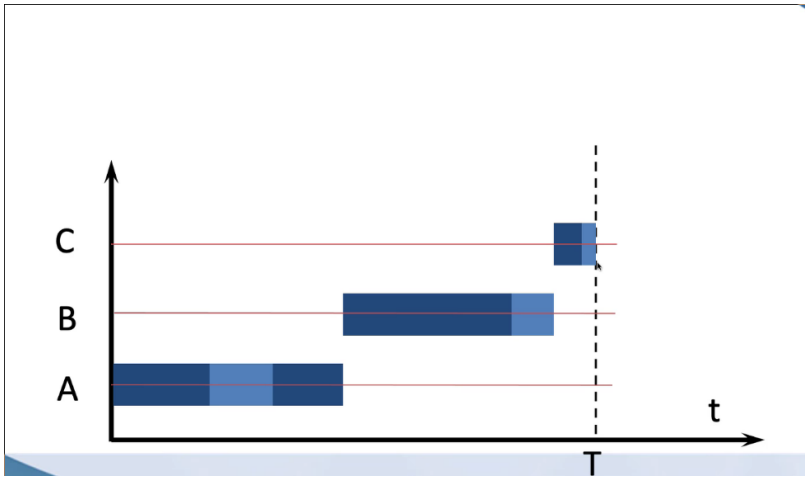


2.15 Politiche utilizzate dallo scheduler

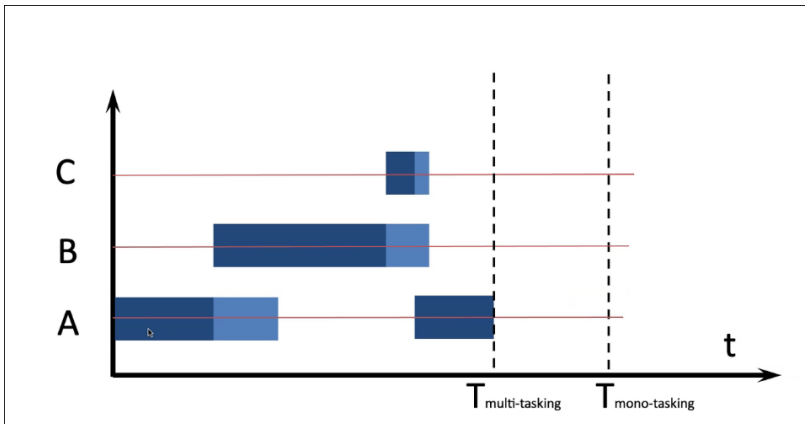
- **Preemtive** → la CPU in uso può essere tolta e passata a un altro in qualsiasi momento;
- **Non Preemtive** → una volta che il processo ha ottenuto l'us della CPU non può essere interrotta fino a che lui stesso non la rilascia;

2.16 Sistema mono e multi tasking

La gestione della CPU (mantenere in maniera efficiente il suo uso) è un obiettivo del sistema operativo. Nel caso di un sistema mono-tasking avremo:



Nel caso di un sistema multi-tasking avremo:



(*In blu scuro il tempo di utilizzo CPU e in chiaro il tempo di attesa di eventi esterni*)

2.17 Throughput (utilizzo della CPU)

Si ha dal rapporto di Tp/Tt dove:

- Tp = tempo dedicato dalla CPU alla esecuzione dei processi utente;
- Tt = tempo totale di osservazione

Si ha un'alta efficienza con un throughput tendente ad 1

2.18 Overhead

Tempo impiegato dal SO per trasferire il controllo da un programma ad un altro

3 Lezione del 12-03

3.1 Sincronizzazione dei processi

Uno dei compiti che spetta al SO è la sincronizzazione dei processi. Più processi su una macchina possono essere:

- In **competizione**
  - se si cerca di usare la stessa risorsa contemporaneamente;
- In **cooperazione**
  - quando uno ha bisogno dell'altro per evolvere;

La sincronizzazione avviene grazie a variabili condivise dette **semafori** o a scambio di messaggi tra processi

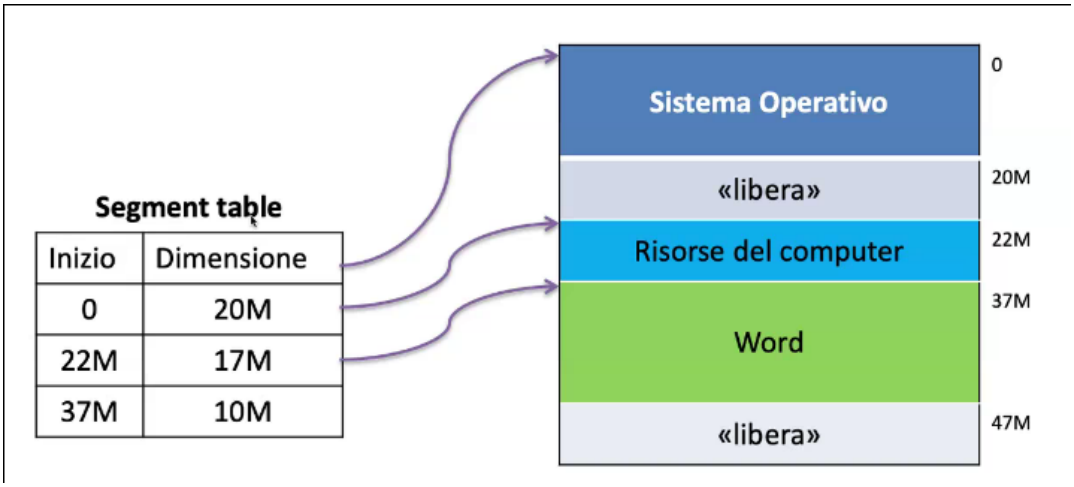
Ogni processo per essere eseguito deve sessere caricato in memoria. Nei sistemi multitasking permette di caricare più processi contemporaneamente.

Il gestore della memoria deve:

- Tenere traccia di quali parti della memoria sono usate e da chi;
- Decidere quali processi caricare quando diventa disponibile spazio in memoria;
- Allocare e deallocare memoria.

Per ogni blocco di memoria il SO mantiene un **descrittore** con le informazioni essenziali. La memoria fisica è suddivisa in un determinato numero di aeree (segmenti)

Ogni entry nella **segment table** contiene l'indirizzo di base di un segmento e la sua lunghezza



Lo svantaggio principale è la frammentazione (la memoria non viene sfruttata completamente). Per ovviare a ciò esiste un processo di compattamento → traslazione dei programmi.

Politiche di gestione dei frammenti

- **First-fit** → alloca il primo frammento libero sufficiente;
- **Best-fit** → alloca il più piccolo frammento libero sufficiente;
  - ricerca sull'intera lista e produce frammenti piccoli
- **Worst-fit** → alloca il più grande frammento
  - ricerca sull'intera lista e produce frammenti grandi

Nei moderni SO si utilizza la **paginazione dei segmenti**, ossia un segmento viene realizzato tramite un insieme di pagine

La suddivisione della memoria centrale in blocchi di dimensioni fisse (pagine) tipicamente 2KB o 4KB.

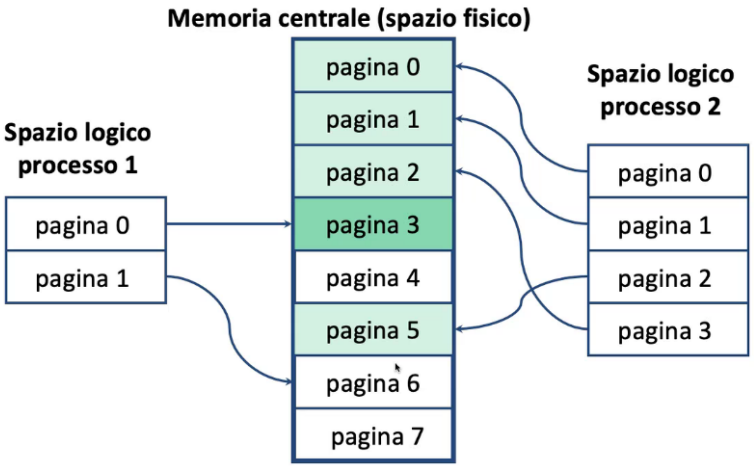
L'immagine di un processo viene suddivisa in pagine caricate all'interno della memoria principale in modo indipendente, anche in posizioni non contigue.

Il SO gestisce una **Page Table** che memorizza per ogni pagina (spazio logico) del processo e la sua posizione in memoria centrale (spazio fisico)

Frammentazione interna ed esterna

- **Frammentazione interna** → Con i blocchi di dimensioni fisse, all'interno di ogni blocco ci sarà uno spazio non utilizzato

- **Frammentazione esterna** → con i blocchi di dimensioni variabili, dopo un certo numero di allocazione e deallocazione di processi, si formeranno un certo numero di porzioni di memoria libere (buchi) di dimensioni insufficienti a contenere un processo



Se durante l'esecuzione del processo, la CPU fa riferimento ad un indirizzo che appartiene ad una pagina non caricata in memoria centrale (page fault), il SO provvede a mettere il processo in stato d'attesa e a far partire il caricamento della pagina richiesta (swap), dopodichè il processo può riprendere la sua esecuzione.

Nella paginazione esiste ancora il problema della frammentazione, che è però limitato al fatto che l'ultima pagina di ogni processo è solo parzialmente occupata. Mediamente si può considerare uno spreco pari a circa 1/2 pagina per ogni processo

3.2 Comunicazione con i dispositivi

Il gestore delle periferiche permette la comunicazione con le varie periferiche. Particolari programmi detti device driver gestiscono le operazioni di `in/out` tra le varie periferiche collegate alla macchina.

Le funzioni svolte dai device driver sono:

- Rendere trasparenti le caratteristiche fisiche tipiche di ogni dispositivo;
- Gestire la comunicazione dei segnali verso i dispositivi;
- Gestire i confiltti, nel caso in cui due o più task vogliono accedere contemporaneamente allo stesso dispositivo

3.3 File system

Gestisce le memorie di massa (organizza logicamente i dati e le possibili operazioni su di essi)

3.4 La shell dei comandi

L'interprete dei comandi o shell è quella parte più esterna di un SO che riceve ed elabora le istruzioni impartite da un utente Il SO presenta due tipi di intefaccia:

- testuale;
- grafica (diversa tra mobile e pc);

3.5 La memoria virtuale

Sistema `SW/HW` in grado di simulare uno spazio di memoria centrale maggiore di quello fisicamente presente

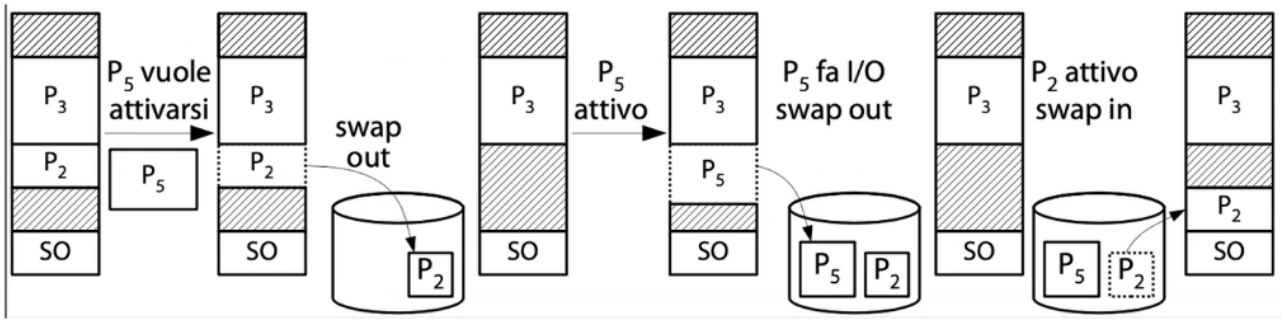
Rappresenta in modo astratto e logico la memoria cetrale di un calcolatore

Memory management unit

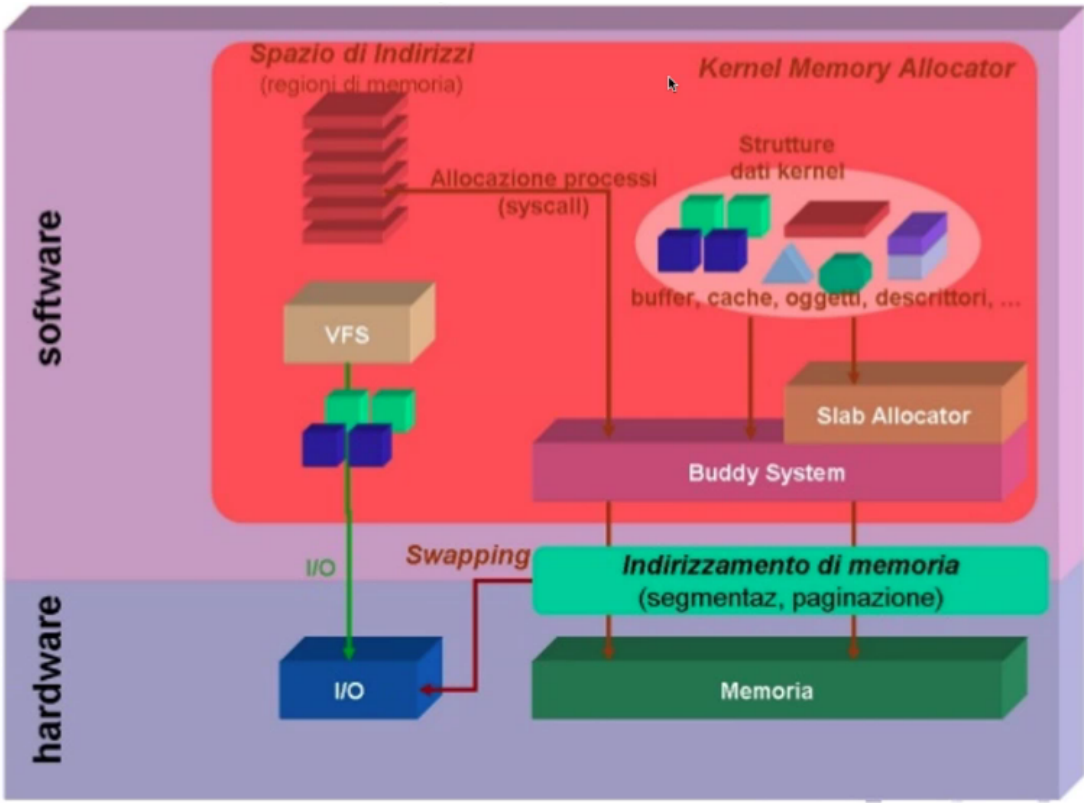
Gestisce le richieste di memoria delle applicazione traducendole in istruzioni da impartire all'unità HW di gestione della memoria

Vantaggi:

- Esecuzione concorrente dei processi;
- Esecuzione di applicazioni con richieste di memoria maggiori della memoria fisica;
- Condivisione da parte dei processi di una singola immagine di memoria di una libreria o del programma;
- Programmi rilocati



3.6 Memoria virtuale in Linux



Buddy System → tecnica di allocazione dinamica della memoria che divide la memoria in partizioni per soddisfare una richiesta di memoria nel miglior modo possibile.

Slab allocation → tecnica di allocazione della memoria per una migliore allocazione degli oggetti. Riduce la frammentazione causata dalla allocazione e della deallocazione

Kernel Memory Allocator

È la componente del kernel che cerca di soddisfare richesta di aree di memoria provenienti dal kernel stesso e dalle applicazioni

- Il KMA risponde a richieste di memoria provenienti da:
  - altri sottosistemi del kernel per uso del SO;
  - dai programmi utenti per mezzo di system call.
- Un KMA dovrebbe avere queste caratteristiche:
  - essere veloce;
  - minimizzare sprechi di memoria;
  - ridurre la frammentazione;
  - cooperare con gli altri sottosistemi di memoria per ottenere e rilasciare pagine

3.7 Spazio di indirizzi di un processo

Lo spazio di indirizzi virtuali che ogni processo può usare è diverso dall'insieme degli indirizzi fisici che usa ad ogni istante

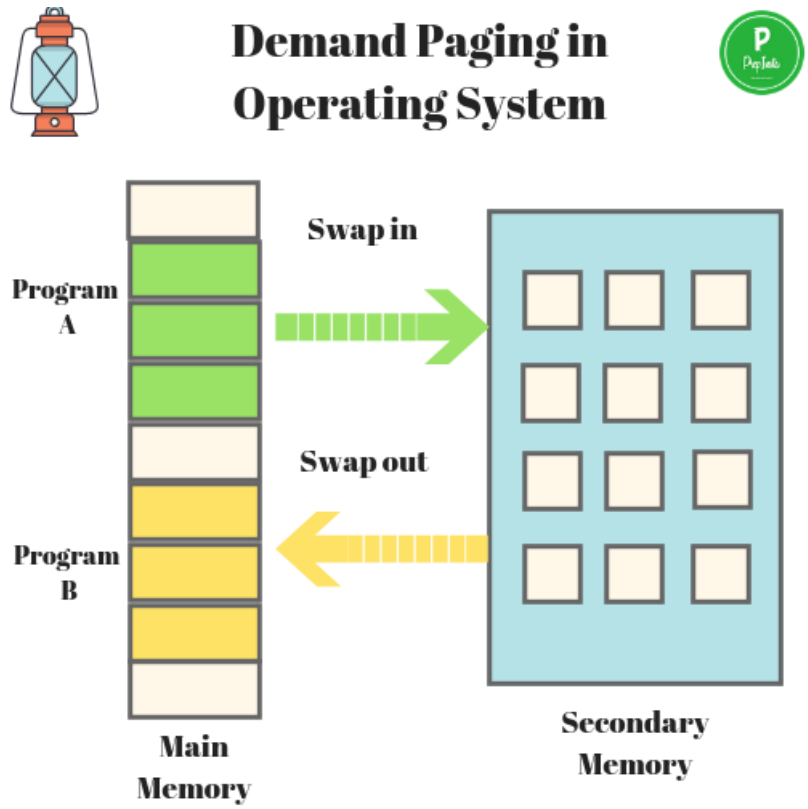
Il kernel rappresenta lo spazio di indirizzi come lista di descrittori di regioni di memoria

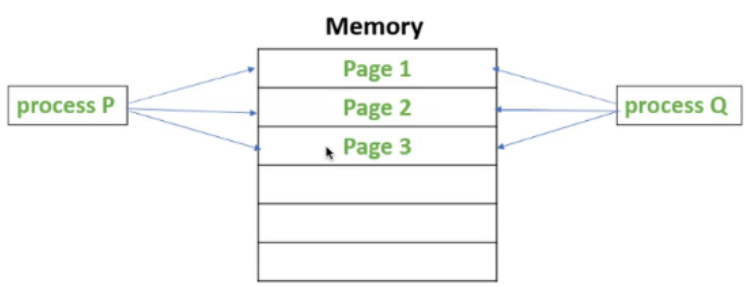
Alla creazione di un processo il kernel gli associa uno spazio di indirizzi che include regioni di memoria per:

- codice eseguibile;
- i dati inizializzati (e non) del programma;
- lo stack del programma;
- il codice eseguibile e i dati delle librerie condivise;
- lo heap (memoria dinamicamente richiesta dal processo);
- mapping in memoria di file usati dal programma

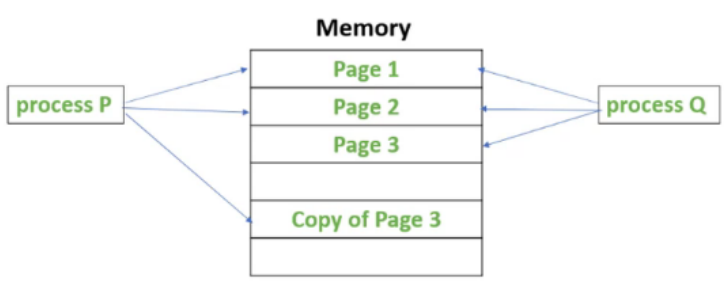
I moderni SO Unix usano strategie efficienti di allocazione di pagine basate sul Demand Paging e Copy-on-Write

Demand Pagin e Copy-on-Write a confronto





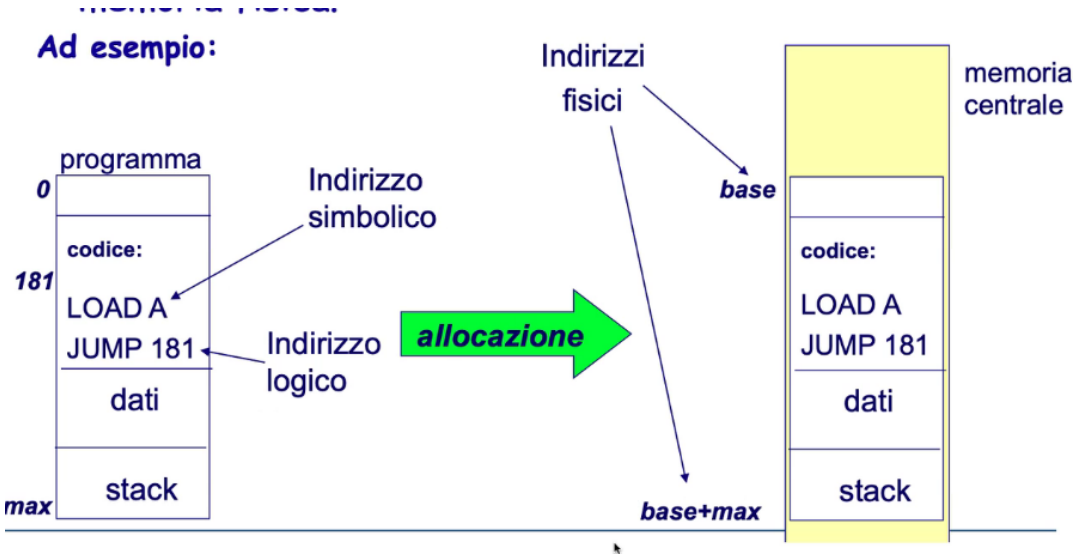
Before process P modifies Page 3



After process P modifies Page 3

Con la fork padre e figlio non lavorano sullo stesso spazio di indirizzamento, ma il figlio lavora su una copia dei dati del padre (a differenza dei thread che condividono la stessa memoria)

Ogni processo dispone di un proprio spazio di indirizzamento logico `[0,max]` che viene allocata nella memoria fisica



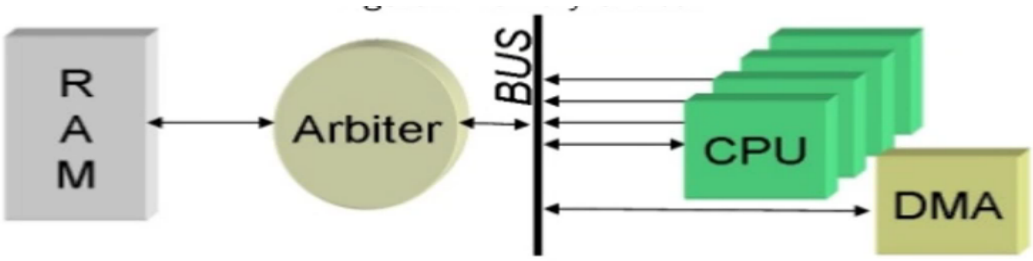
4 Lezione del 15-03

4.1 Swapping

Meccanismo HW e SW che consente di estendere la memoria centrale utilizzando spazi aggiuntivi su supporti fisici. Lo swapping è basato sulle pagine (swapped-out)

4.2 Memory Arbiter

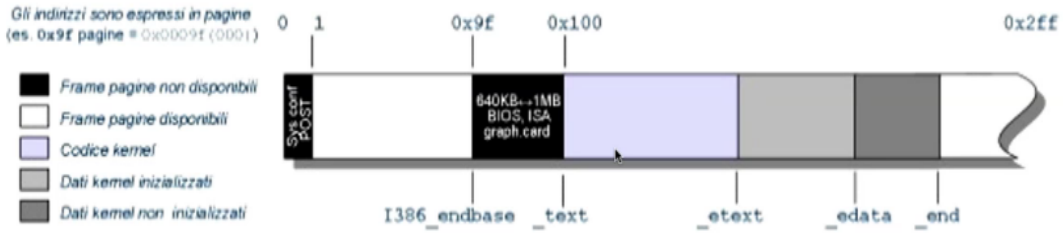
Un circuito HW che serializza (disciplina) gli accessi write-read alla ram da parte delle CPU (o dalla CPU e del DMA (accedere direttamente alla memoria interna per scambiare dati, in lettura e/o scrittura, senza coinvolgere CPU))



4.3 La RAM nei sistemi UNIX

È divisa in 2 sezioni:

- pochi mb per immagazzinare l'immagine del kernel;
- Il resto viene gestito dal sistema di memoria virtuale come memoria dinamica
  - Viene usata dal kernel per una serie di operazioni diverse



Tutto l'insieme dei meccanismi HW e SW per tradurre l'accesso ad un indirizzo di memoria virtuale (logico) nel corrispondente acccesso alla locazione fisica in RAM.

Il SO non deve occupari di tutti gli aspetti dell'accesso alla memoria fisica (nei moderni microprocessori esiste supporto HW per la traduzione e meccanismi hHW per l'individuazione e segnalazione di errore)

4.3.1 Risultati desiderati

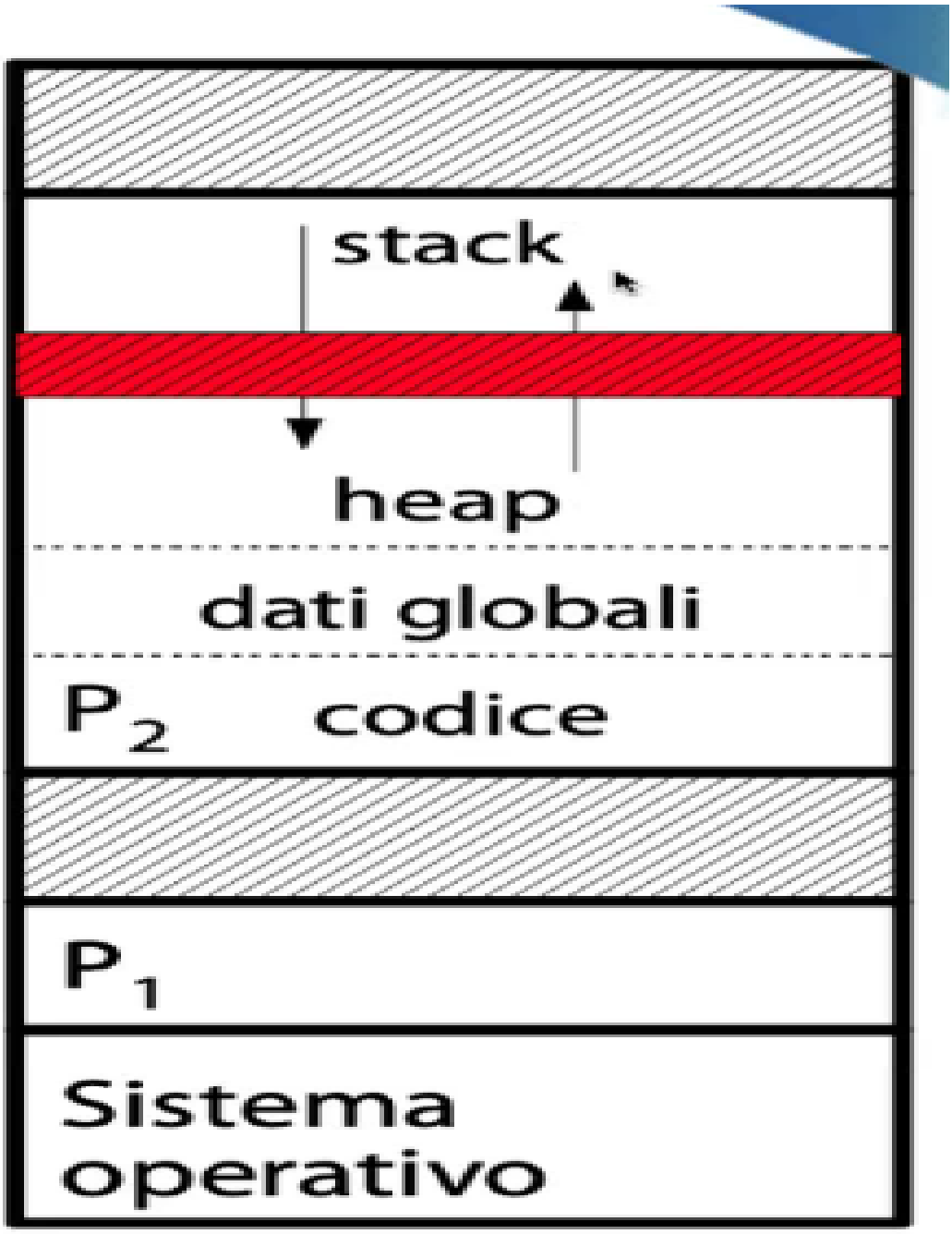
- La partizione di un processo deve essere sempre collocata in una porzione di memoria continua → swap di porzioni mirate della memoria di un processo
- Necessità di tenere lontani heap e stack
- Si deve impedire ad un programma l'accesso alla memoria di un altro programma
- Si deve impedire o limitare il buffer overflow

4.3.2 Stack & heap

Stack → heap: le strutture dati in cima all'heap contengono valori dello stack → corruzione dei dati sull'heap

Heap → stack: gli ultimi record di attivazione sono sovrascritti con valori provenienti dall'heap → il programma salta ad una locazione più o meno casuale (buffer overflow)

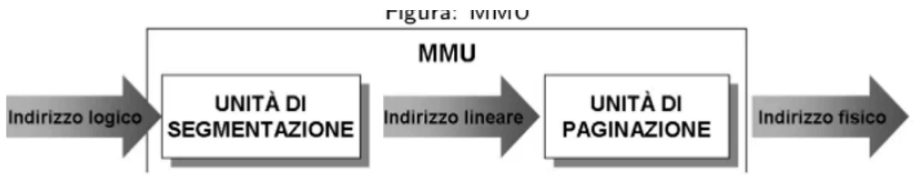
Se i dati sull'heap sono scelti bene, è possibile eseguire codice arbitrario

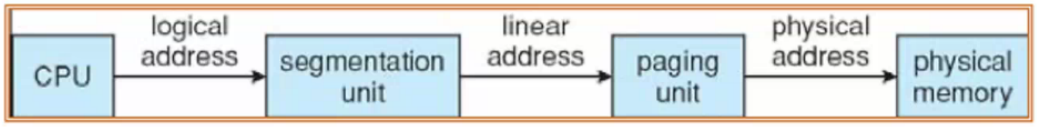

stack
heap
dati globali
P2 codice
P1
Sistema operativo

4.4 Memory Management Unit (MMU)

Componente della CPU che traduce indirizzi virtuali in fisici:

- Usa una circuitistica HW detta Unità di Segmentazione per tradurre indirizzi logici in indirizzi lineari
- Una una circuitistica HW detta Unità di Paginazione per tradurre indirizzi lineari in indirizzi fisici





4.5 Segmentazione

Schema di gestione della memoria che implementa una visione del programma per "blocchi funzionali"

4.6 Indirizzo logico

Un numero binario a 48 bit formato da un selettore di segmento (16 bit) e un offset (32 bit)

4.6.1 Segment selector

Identifica il segmento a cui appartiene l'indirizzo di memoria. Diviso in

- TI (1 bit) → indica in quale tabella è immagazziona il descrittore di segmento
  - GDT → tabella dei descrittori globali
  - LDT → tabella dei descrittori locali
- indice (13 bit) → posizione descrittore in tabella (GDT o LDT);
- RPL (2 bit) → flag che indica il CPL (current privilege level) che aveva la CPU quando il segmento viene caricato;
- i 6 registri di segmento (cs, ss, ds, es, fs, gs) contengono i selettori di segmento
  - i rimanenti bit non sono usati (cs che neusa 2 per il CPL)
    - cs → code segment;
    - ss → stack segment;

* ds → data segment;
* es → extra segment;
* fs, gs → inseriti per operazioni future;

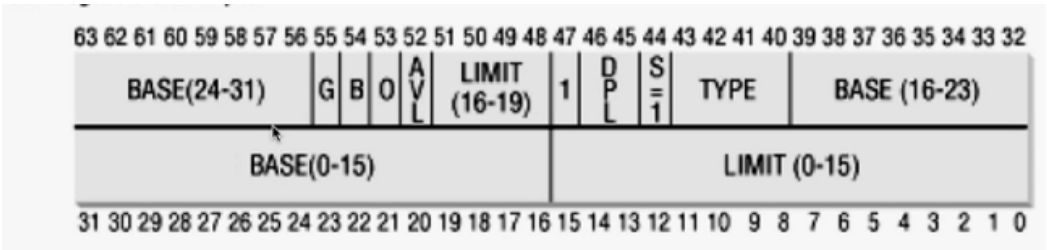
4.6.2 Offset

Indica la posizione dell'indirizzo rispetto all'inizio del segmento

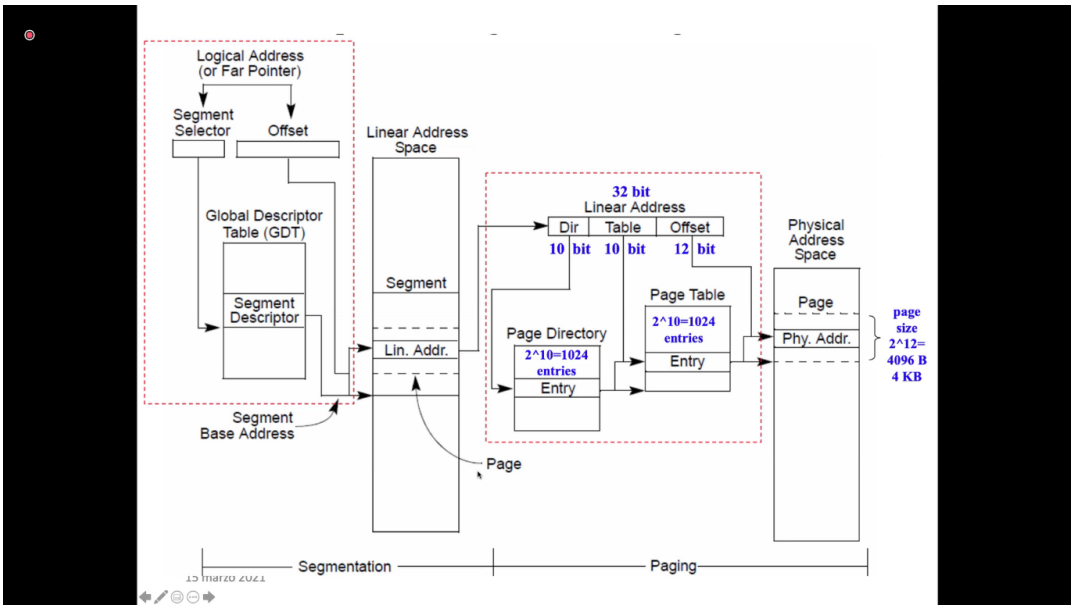
4.6.3 Descrittore di segmento

Dato di 64 bit che descrive le caratteristiche del segmento:

- BASE → indirizzo lineare base del primo byte del segmento;
- LIMIT → dimensione del segmento;
- TYPE → tipo di segmento;
- DPL (Descriptor Privile Level) → livello minimo di privilegi per accedere al segmento;
- bit 47° → 1 se presente in memoria, 0 altrimenti



4.6.4 Traduzione degli indirizzi logici in indirizzi fisici



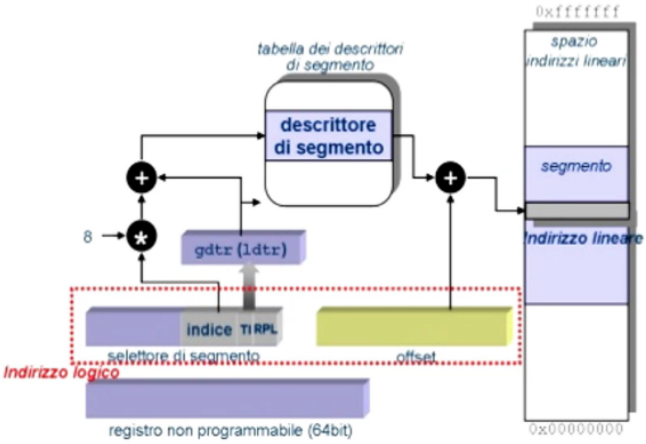
Ogni registro ha un registro non programmabile (con lo scopo di velocizzare la traduzione)

In esso è caricato il descrittore del segmento quando, per la prima volta, è caricato il selettore di segmento nel corrispondente registro segmento

Se il contenuto di un registro segmento cambia (flush), cioè un nuovo selettore è caricato, la segmentazione passa per la GTD (o LDT). Nel caso contrario (segmentazione diretta) non passa per la GTD (o LDT)

1. il valore `TI` determina se il descrittore si trova in GDT o LDT
2. l'indirizzo base della GDT (LDT) è nel registro `gdtr` (`ldtr`)
3. l'indice×8+ind.base = posiz. descr. in GDT (o LDT). Copia descr. in r.n.p.
4. dal descrittore si ricava l'indirizzo (lineare) base del segmento
5. ind. base segmento + offset = indirizzo lineare tradotto

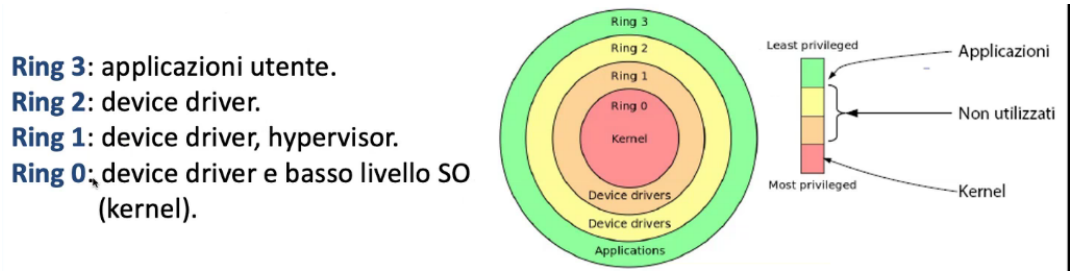
Figura: segmentazione: accesso in GDT



4.6.5 Privilegi di esecuzione

Meccanismo di difesa da istruzioni o accessi alla memoria causati da malfunzionamenti HW o comportamenti malevoli

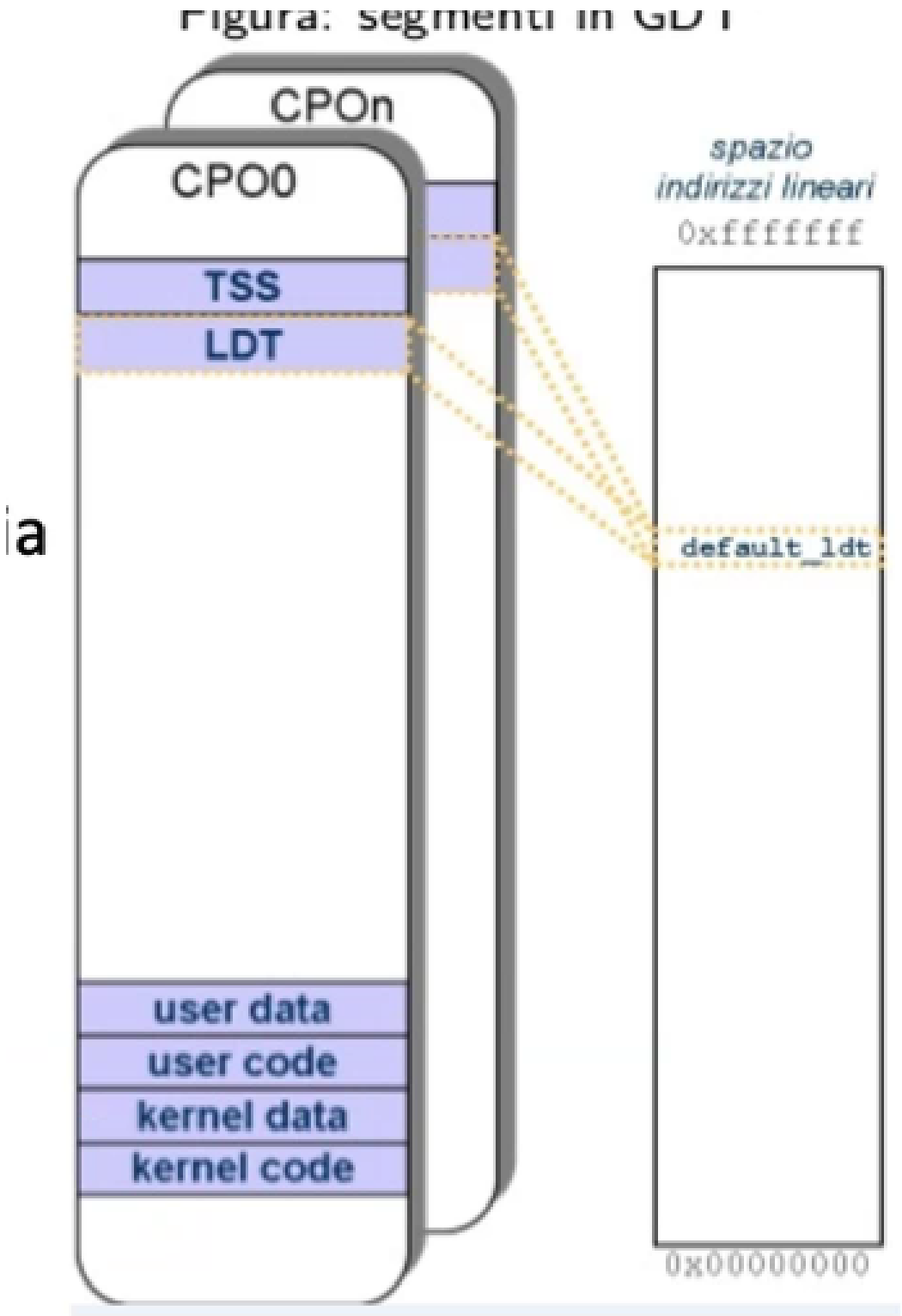
La CPU è dotata di 4 modi di utilizzo disposti ad anello



4.6.6 Tabelle dei Descrittori di Segmento

Vengono immagazzinati in GDT o LDT. Queste tabelle sono immagazzinate in memoria RAM.

I registri gdtr e ldtr contengono rispettivamente gli indirizzi fisici base della GDT e LDT



4.7 Segmentazione in sistemi LINUX

Linux usa principalmente la paginazione e in modo limitato la segmentazione (si limita il numero di flush)

4.7.1 I segmenti in Linux

1. Kernel code Segment Corrisponde ad un segmento di codice del kernel ed il suo segment selector è definito con la macro `__KERNEL_CS`
2. Kernel Data Segment Corrisponde ad un segmento di dati del kernel ed il suo segment selector è definito con la macro `__KERNEL_DS`
3. User Code Segment Corrisponde ad un segmento di codice condiviso da tutti i processi utente ed il suo segment selector è definito con la macro `__USER_CS`
4. User Data Segment Corrisponde ad un segmento di dati condiviso da tutti i processi utente ed il suo segment selector è definito con la macro `__USER_DS`

4.7.2 Spazio degli indirizzi lineari

Intervallo di indirizzi lineari da 0 (0x0000000) a 4G (0xFFFFFFF).

La dimensione massima dello spazio di indirizzi è determinata dall'architettura della CPU

4.7.3 Pagina di indirizzi lineari

Intervallo di indirizzi lineari di dimensione fissa (in Linux 4K); l'indirizzo base dell'intervallo è multiplo della dimensione.

4.7.4 Segmentazione vs Paginazione

Entrambi sono ridondanti → entrambi `partizionano/assegnano` la memoria fisica tra processi.

- La segmentazione assegma intervalli di indirizzi lineari a diversi ai processi;
- La paginazione mappa un intervallo di indirizzi lineari su intervalli di indirizzi fisici diversi

4.7.5 Page

Un blocco di dati che può essere scritto su un frame pagina.

Questo termine si riferisce sia alla pagina di indirizzo che alla pagina di dati memorizzati in RAM in corrispondenza.

Una frame pagina memorizza esattamente il blocco di dati (o istruzioni) contenuti in una pagina (di indirizzi lineari).

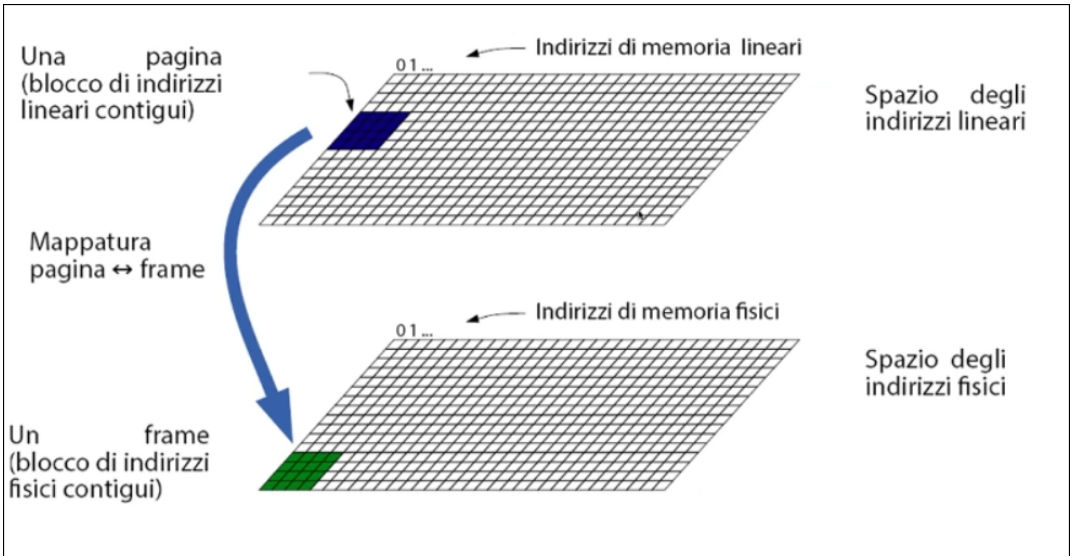
La paginazione mappa pagine di indirizzi lineari su frame pagine; una pagina è in RAM o in disco

1. Page Frame Area fisica (intervallo di locazioni contigue) di memoria RAM di dimensione fissata
2. Paging Unit Circuitistica della MMU che converte indirizzi lineari in fisici. Nella stessa pagina, indirizzi lineari contigui sono paginati su indirizzi fisici contigui.

   L'ultimo indirizzo di una pagina e quello della pagina successiva possono essere mappati su indirizzi fisici non contigui

   I permessi di accesso e i privilegi associati ad una pagina valgono per tutti gli indirizzi in essa contenuta.

   La paginazione verifica anche che la richiesta di accesso abbia privilegi e modalità compatibili con quelli associati alla pagina di appartenenza



3. La tabella delle pagine Ha una dimensione alta (viene mantenuta in RAM).

   Nei processori intel l'indirizzo fisico iniziale della tabella è salvato nel registro di controllo CR3.

   Come detto essendo gigantesca la sua dimensione si fa uso di uno **schema di paginazione gerarchico** → ciascun elemento della tabella delle pagine punta all'inizio di una tabella più piccola (che puntano ai frame fisici)

4.7.6 Tipi di Paginazione

- A 2 livelli (sistemi a 32 bit);
- A 3 livelli (in quanto il precedente non è più sufficiente);
- A 4 livelli (usati dai sistemi Linux), con 4 tipi di tabelle
  - Page Global Dir (PGD);
  - Page Upper Dir (PUD);
  - Page Middle Dir (PMD);
  - Page Table (PT).

4.7.7 TLB

Cache HW che velocizza la traduzione di indirizzi lineari in fisici

La prima volta che un indirizzo lineare è tradotto, l'indirizzo fisico corrispondente è registrato in un elemento della TLB. Per usi successivi dello stesso indirizzo linearre si userà l'indirizzo fisico presente in TLB.

Ogni CPU ha la propria TLB → sincronizzazione non necessaria:

- Processi che seguono su diverse CPU possono usare stessi indirizzi logici corrispondenti a indirizzi fisici diversi;
- TLB funziona da cache per gli elementi della Page Table;
- Se cambia l'indirizzo fisico di una frame pagina (swapping), l'elemento della TLB corrispondente è invalidato;
- Se cambia il valore del registro CR3 (PD) tutta la TLB viene invalidata

5 Lezione del 19-03

5.1 La programmazione di sistema

Consiste nell'utilizzare l'interfaccia di system call fra il kernel e le applicazioni che girano sotto UNIX.

5.1.1 System call

Per i programmi C, le system call sono come funzioni. Ogni system call ha un prototipo:

- `pid_t` (0 se padre, altro valore se figlio)

Tra le system call ricordiamo:

- getpid;
- getppid;
- getgrp;

Che forniscono degli attributi dei processi (PID, PPID, gruppo etc)

- fork → crea un processo figlio duplicando il chiamante;
- exec → trasforma un processo sostituendo un nuovo programma nello spazio di memoria del chiamante;
- wait → permette la sincronizzazione dei processi;
- exit;

5.1.2 Fork

Il prototipo è `pid_t` *fork()*, dove:

- `pid_t` è definito negli include di sistema.

Il valore restituito da fork può essere usato per distinguere tra processi genitore e processo, infatti al genitore viene restituito il PID del figlio, al figlio viene invece restituito 0. Restituisce 0 al figlio e pid al padre, oppure -1 (solo al padre) in caso di fallimento

5.1.3 Processo Zombie

Un processo diviene zombie dopo essere terminato e prima che il processo padre abbia eseguito una *wait* su quel figlio

5.1.4 Esempio

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main(){
    pid_t pid;
    int status;
    pid = fork();
    if (pid)
        sleep(2);
        pid = wait (&status);
        if (WIFEXITED (status)) {
            printf ("stato %d\n", WEXITSTATUS(status));
        } else{
            printf ("Processo %d, figlio. \n ", getpid());
            _exit(17);
        }
}
```

5.2 Gestione dei processi in Linux

Si definisce processo una qualsiasi istanza di programma in esecuzione.

Supporto di multithreading, attraverso i lightweight process (queste condividono le risorse) → in questo modo vengono supportati i thread.

5.3 Process Descriptor

Il kernel monitora costantemente l'attività dei processi mediante una struttura di tipo `task_struct` che prende il nome di Process Descriptor

Questo descrittore contiene una serie di puntatori a struttre secondarie (che contengono le relative informazioni)

Il kernel contiene la lista dei processi in una lista doppiamente linkata chiamata task list

5.4 Stati da un processo

- `TASK_RUNNING`;
- `TASK_INTERRUPTIBLE`;
- `TASK_UNINTERRUPTIBLE`;
- `TASK_STOPPED`;
- `TASK_TRACED`;
- `EXIT_ZOMBIE`;
- `EXIT_DEAD`.

5.5 Identità di un processo

Qualunque processo in Linux è caratterizzato da un process descriptor ed il puntatore a tale struttura prende il nome di process descriptor pointer. Linux identifica i processi assegnando loro un Process ID (PID) che viene memorizzato nel campo `pid_field` del process descriptor.

I PID vengono assegnati in ordine crescente e riutilizzati solo dopo aver raggiunto il valore massimo. Linux tiene traccia dei PID nuovamente disponibili in un `pidmap_array`

5.6 Relazioni fra processi

I principali sono:

- Relazione `padre/figlio;`
- Relazione processi fratelli.

Ma ne esistono molti altri in base al condizione che andiamo a verificare.

5.7 Le liste di processi RUNNING

Nelle precedenti versioni di linux, la selezione del processo da eseguire avveniva scorrendo una *runqueue* alla ricerca del processo con priorità più alta.

Le attuali versioni implementano un algoritmo di selezione in tempo costante, mantenendo k liste di processi, dove k rappresenta la priorità (un valore intero tra 0 e 139)

5.8 I file hash

Si servono di una funzione di hash o funzione di randomizzazione, tale che applicata al valore di uno dei campi restituisce l'indirizzo del blocco in cui è memorizzato il record. Consentono operazioni di ricerca molto efficienti.

Per file interni, l'hash è implementato con una tabella di hash.

5.8.1 Funzione di accesso h

Definita come:

$$h{:}K \rightarrow \{0,1,2,\ldots,\ m\text{-}1\}$$

Dove:

- K → insieme dei valori distinti che il campo chiave può assumenre;
- m → dimensione del vettore in cui si intende memorizzare gli elementi della tabellla

5.8.2 Divisione

Definita come:

$$h(k) = k \bmod |T|$$

Presenta una bassa complessità, ma può generare fenomeni di agglomerazione

5.8.3 Risoluzione delle Collisioni

Esistono diverse strategie per la risoluzione delle collisioni:

- Concatenamento → alla i-esima posizione della tabella è associata la lista degli elementi tali che *h(k)=1*;
- Indirizzamento aperto → tutti gli elementi sono contenuti nella tabella.

La scelta della strategia influisce sull'efficienza delle operazioni di inserimento, ricerca e cancellazione.

5.8.4 Tabella Hash per la runqueue

Il Kernel mantiene quattro tabelle hash, che gli permettono di collegare il PID di un processo al suo process descriptor

Le collisioni sono gestite mediante liste concatenate

5.9 Organizzazione dei processi

La runqueue contiene solo i processi nello stato `TASK_RUNNING`.

I processi nello stato `TASK_STOPPED` `EXIT_DEAD` `EXIT_ZOMBIE` non vengon raggruppate in nessuna lista.

I processi che si trovano nello stato di `TASK_INTERRUPTIBLE` e `TASK_UNINTERRUPTIBLE` vengon raggruppati a seconda dell'evento per cui sono stati accodati

Poichè lo stato di un processo non è sufficiente a decidere la politica di risveglio da adottare si fa uso delle wait queue

5.9.1 Wait queue

Usate dal kernel per gestire diverse situazioni:

- Gestione degli interrupt;
- Sincronizzazione dei processi;
- Timing.

Sono implementate mediante liste doppiamente concatenate, i cui elementi puntano a process descriptors. Sono fornite inoltre di uno spinlock poichè possono essere utilizzate sia delle principali funzioni del kernel, che dai gestori degli interrupt

5.9.2 Struttura di una Wait queue

Il campo `task_list` contiene il puntatore alla lista di processi che attendono nella coda. La decisione di risvegliare un processo in attesa in una coda dipende da diversi fattori. In particolare bisogna valutare se esso attende per una risorsa che può essere acceduta in modo esclusivo.

Esistono due tipi di processi sleeping:

- Esclusivi $\rightarrow$ identificati dal campo flags impostato a 1;
- Non esclusivi $\rightarrow$ identificati dal campo flags impostato a 0;

5.10 Duplicazione delle risorse

I SO Unix si basano sulla generazione a catene di processi per servire le richieste dell'utente. Unix tratta i processi allo stesso modo.

Le risorse di un processo sono duplicate e assegnate ai figli. Questo modalità è inefficiente.

Nei moderni sistemi Linux questo problema viene risolto attraverso diverse tecniche:

- Copy-or-Write;
- Lightweight processes (threads);

5.10.1 Chiamate di sistema in Linux

Ne esistono 3:

- fork() → valore di ritorno
  - nel contesto di esecuzione del padre il PID del processo figlio;
  - nel contesto di esecuzione del figlio ritorna 0;
  - padre e figlio non condividono lo spazio di indirizzi
- vfork() → analogo al fork;
- clone) → valore di ritorno
  - PID del processo creato;
  - -1 in caso di fallimento.
  - padre e figlio condividono le risorse indicate

Tutte e 3 invocano la stessa funzione di servizio `do_fork()`

5.11 Bomba fork

Attacco di tipo denialof service contro un computer che utilizza la funzione fork.

Es in bash

```
:() { : | : & };:
```

5.12 Terminazione di un processo

Un task termina su sua richiesta (sincrono) quando:

- finisce di eseguire il proprio codice
  - chiamata esplicita o implicita della funzione di sistema *exit()* (o `pthread_exit()`)

Un task non termina su sua richiesta (asincrono) quando:

- riceve segnali di terminazione;
- avviene un'eccezione.

In tutti i casi è il Kernel che ne forza la terminazione:

- rilasciando tutte le risorse del task;
- rimuovendo il task dalle liste del sistema e deallocandole il descrittore + KMS

5.13 I thread

Rappresenta uno dei possibili sottoprocessi eseguibili all'interno di esso. Sono quindi parti di un processo che vengono eseguite in maniera concorrente ed asincrona

Nei thread vengono condivise le medesime informazioni di stato, la memoria ed altre risorse di sistema

La differenza è il meccanismo di attivazione:

- l'operazione di attivazione di un processo è un operazione onerosa;
- l'operazione di attivazione di un thread richiede meno risorse.

5.13.1 Kernel-level e user-level thread

Thread a livello utente

Sono gestiti come uno strato separato sopra il nucleo del SO, e realizzati tramite una libreria di funzioni per la creazione, lo scheduling, e la gestione, senza alcun intervento diretto del nucleo

Thread a livello del nucleo

Gestiti direttamente dal SO

Distinguiamo 3 tipi di modelli:

1. Modello da molti a uno Fa corrispondere molti thread a livello utente a un singolo thread al livello del nucleo
2. Modello da uno a uno Fa corrispondere ciascun thread a livello utente con un thread al livello del nucleo.
3. Modello da molti a molti Fa corrispondere più thread a livello utente con un numero minore o uguale di thread al livello del nucleo

5.13.2 Clone()

Esiste la possibilità attraverso la system call clone() di generare thread. Questa accetta come parametro un insieme di flag che definiscono quante e quali risorse del processo genitore debbano essere condivise dal processo figlio, come:

- `CLONE_FS` → le informazioni sul file system sono condivise;
- `CLONE_VM` → condivisione dello stesso spazio di memoria;
- `CLONE_SIGHAND` → condivisione dei gestori dei segnali;
- `CLONE_FILES` → condivisione dell'insieme dei file aperti

5.13.3 Pthreads

Lo standard POSIX (Portable Operating System Interface for UniX) → definisce l'API per la creazione e la sincronizzazione dei thread.

Non si tratta di una realizzazione ma di una definizione del comportamento dei thread.

I progettisti di SO possono realizare le API così definite come meglio credono